Introduction to Streaming Systems and Architectures

In this train, we'll explore the fundamentals of streaming systems and how they are revolutionizing big data analytics. We'll compare traditional batch processing with modern stream processing, identify when and why streaming is needed, examine common streaming architectures, and introduce the core building blocks that make these systems possible.

Learning Objectives

By the end of this train you should be able to:

- Explain the difference between batch and stream processing.
- Understand the use cases and motivation for stream processing.
- Describe key components and tools used in streaming pipelines.
- Recognize the benefits and limitations of real-time data systems.

What Is Stream Processing?

Stream processing is a data processing paradigm that processes data continuously and incrementally as it flows in. It contrasts with batch processing, which handles large chunks of data at scheduled intervals.

Key Characteristics

- **Real-time or near real-time:** Processes data as it arrives.
- **Event-driven:** Often triggered by data events or user interactions.
- **Scalable and responsive:** Designed to handle high volumes with low latency.

Batch Vs Stream: a Side-by-side Comparison

Characteristic	Batch Processing	Stream Processing
Data Handling	Large batches, historical data	Continuous streams, real-time data
Latency	High	Low
Use Cases	Reports, backups, analytics	Fraud detection, monitoring, personalization
Examples	Hadoop, traditional ETL	Kafka, Spark Streaming, Flink

Why Streaming Matters

With the explosive growth of data from IoT devices, social media, and financial systems, businesses need faster ways to react to changes and events. Stream processing helps businesses gain a competitive edge by:

- Powering real-time dashboards and alerts
- Improving customer experiences with live data
- Enabling smarter automation and decision-making

Common Streaming Architectures

There are three main types of architectures commonly used to implement streaming systems:

- **Lambda Architecture:** Integrates both batch and stream processing layers. The batch layer manages historical data and ensures accuracy, while the stream layer handles real-time updates for speed.
- **Kappa Architecture:** Simplifies Lambda by using only stream processing. It reprocesses all data through a single stream processing engine, ideal where real-time data and immutability dominate.
- **Microservices-based Architecture:** Breaks the pipeline into smaller services, each responsible for a specific task (ingestion, transformation, storage, etc.), improving modularity and scalability.

Architecture Comparison Table

Architecture	Data Layers	Best For	Complexity
Lambda	Batch + Stream	Accuracy and real-time insights	High
Kappa	Stream only	Reprocessing and simplicity	Medium
Microservices	Independent processing units	Modular, scalable design	Medium–High

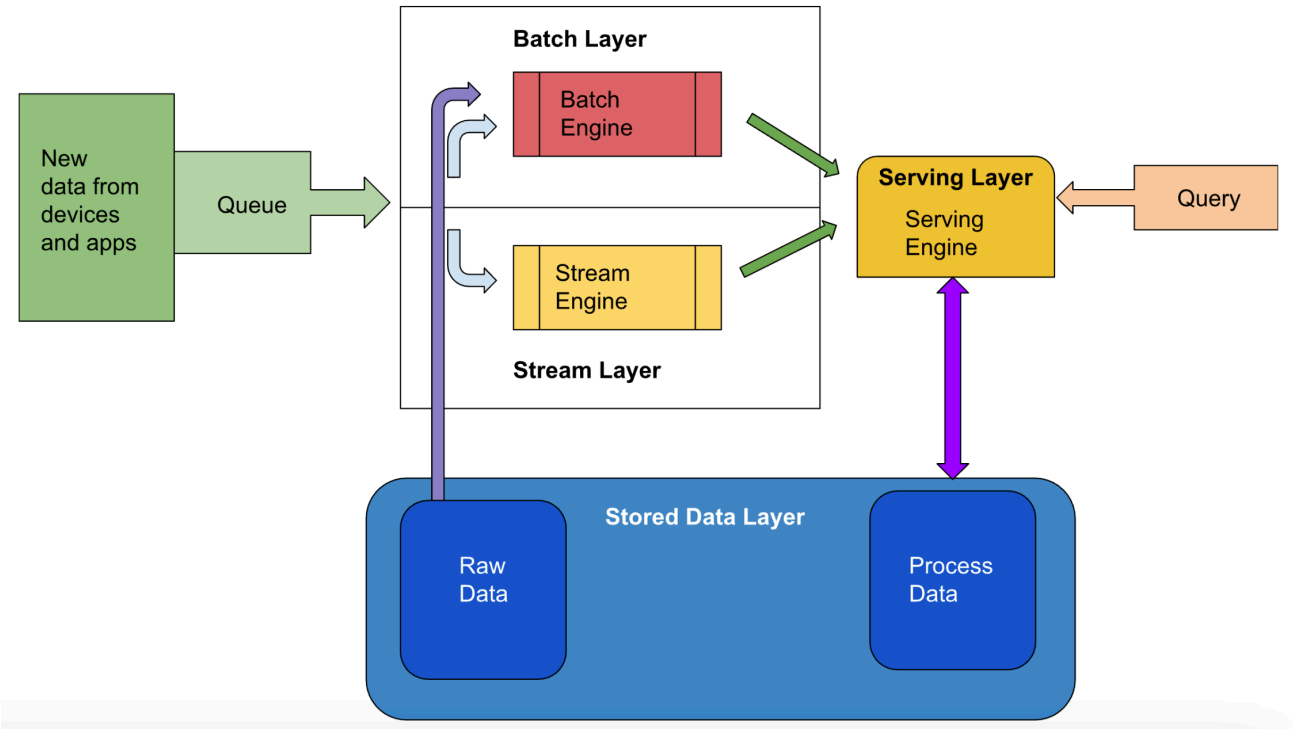


Figure 1: The Lambda Architecture. *This model processes data along two paths: a* ***batch layer*** *(top) handles the complete historical dataset for accuracy, while a* ***stream layer*** *(bottom) provides low-latency, real-time updates. Both outputs are unified in the* ***serving layer*** *to answer queries.*

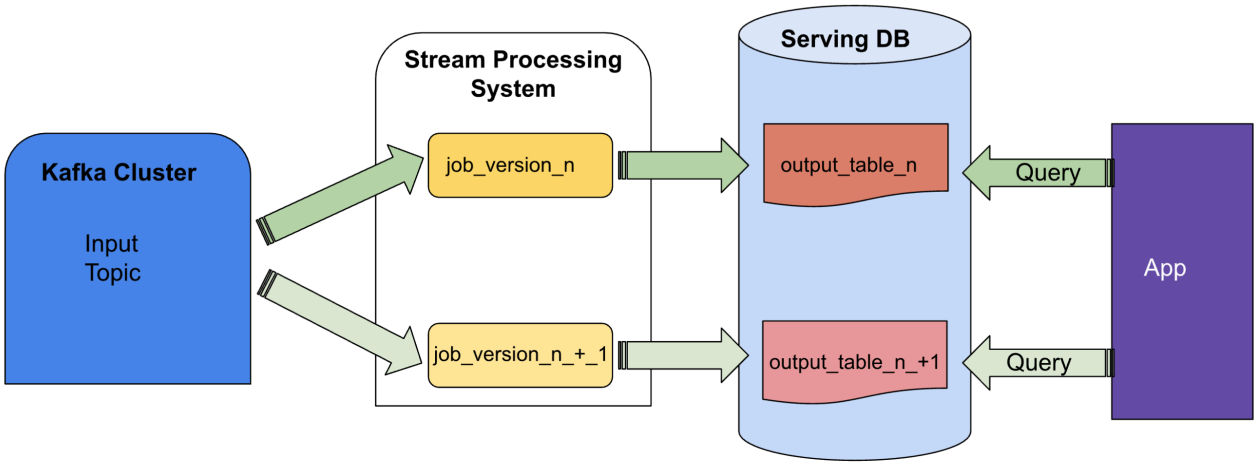


Figure 2: The Kappa Architecture. *This architecture simplifies the data pipeline by removing the batch layer. A single* ***stream processing engine*** *uses an immutable log as the single source of truth, allowing it to generate real-time views and reprocess the entire log history to recalculate data when needed.*

Core Components of Streaming Systems

- **Data Ingestion:** Capturing events and data in real-time (e.g. Apache Kafka)
- **Stream Processing Engine:** Transforming and analyzing data (e.g. Apache Flink, Spark Streaming)
- **Storage:** Persisting results (e.g. S3, HDFS, databases)
- **Serving Layer:** Making outputs available to users or services

Example: Streaming Architecture in Action

Let's take a simplified example of an e-commerce fraud detection system and map it to an architecture:

- **Kafka** serves as the ingestion layer, capturing user transaction events.
- **Flink** acts as the processing engine, performing real-time fraud detection logic.
- **Redis** holds in-memory state for comparisons, such as transaction frequency per user.
- **Power BI** or **Grafana** visualize flagged transactions through dashboards.

This aligns with a Microservices-based architecture, where Kafka and Flink operate as modular services. This allows each component to scale independently and adapt

quickly to evolving business needs.

Conclusion

Stream processing plays a pivotal role in real-time data analytics and modern application responsiveness. As data continues to grow and decisions need to be made faster, understanding stream-based architectures—especially Lambda, Kappa, and Microservices—is essential for any data professional.

Key Takeaways

- **Takeaway 1:** Stream processing handles data in motion and enables real-time insights.
- **Takeaway 2:** Batch is suitable for historical and non-time-sensitive workloads.
- **Takeaway 3:** Tools like Kafka, Flink, and Spark are foundational to streaming systems.
- **Takeaway 4:** Understanding architecture patterns like Lambda, Kappa, and Microservices helps you build scalable systems.